eman ta zabal zazu

Universidad del País Vasco

Euskal Herriko Unibertsitatea

Minería de Datos

Clasificación no supervisada K-Means

Autores: Marcos Cobo Gutiérrez, Ibai Buján Urien, Keneth Sebastián Campos

Fecha: 23/10/2026

Índice

Índice **1**
I. Introducción **2**
A. En qué consiste la tarea 2
B. Qué retos presenta 2
C. ¿Qué han hecho los antecedentes para abordarlos? Análisis DAFO 2
D. ¿Cuál es nuestra propuesta para abordarlos? 2
II. Análisis de datos **3**
A. Descripción cualitativa y fuente de los datos raw 3
B. Pre-proceso: transformación de datos de texto a un vector numérico 3
C. Descripción cuantitativa: número de instancias, instancias repetidas (con los mismos atributos), atributos, talla vocabulario, OOV, missing values, rango de los atributos, … 4
III. Clustering **5**
A. Algoritmo de clustering 5
B. Coste computacional (tiempo y memoria) mediante evidencias experimentales 6
C. ¿Qué medidas de similitud/disimilitud se han contemplado? ¿por cuál nos decantamos? 6
D. ¿Cómo elegir el número de clusters para esta tarea? 7
E. ¿Cómo almacenamos el modelo de clustering? 7
F. Proceso para asignar un cluster a una nueva instancia no vista en la fase de entrenamiento 7
G. ¿Es versátil el software implementado? ¿Se puede extender fácilmente a otros conjuntos de datos? 7
IV. Evaluación **8**
A. Técnicas de evaluación 8
1) Internas 8
2) Externas 9
B. Baseline 9
V. Resultados experimentales **10**
A. Objetivo 10
B. Anexo 10
1) Optimización con Optuna 10
2) Rango de clusters para las pruebas 10
C. Conclusiones 10
VI. Conclusiones y trabajo futuro **11**
A. Nuestra aportación respecto a los antecedentes y las soluciones que hay accesibles 11
B. ¿Cuáles son las debilidades y las fortalezas de nuestro proyecto? 11
C. Líneas de trabajo futuro 11
VII. Bibliografía **12**

I. Introducción

A. En qué consiste la tarea

El proyecto consiste en desarrollar y aplicar un sistema de clasificación no supervisada mediante el algoritmo K-Means combinado con técnicas de procesamiento de lenguaje natural. Su propósito es examinar publicaciones de redes sociales enfocadas en la salud mental utilizando datos basados en la escala clínica C-SSRS **[1]** para clasificar mensajes de usuarios y detectar indicios de malestar emocional, riesgo de suicidio o requerimiento de asistencia.

B. Qué retos presenta

Ruido y alta variabilidad: los textos generados en redes sociales son informales y presentan una dispersión considerable, lo que dificulta su vectorización directa.

Eficiencia en la optimización: las técnicas de vectorización densas requieren de un alto tiempo de cómputo.

Búsqueda de los mejores parámetros: el ajuste manual de los parámetros resulta inoperante ante la gran cantidad de combinaciones posibles.

C. ¿Qué han hecho los antecedentes para abordarlos? Análisis DAFO

Radu et al. **[2]** compararon TF-IDF y Doc2Vec con varios algoritmos de agrupamiento. En sus experimentos, K-Means obtuvo un índice Rand ajustado (ARI) de hasta 0.83 con Doc2Vec, frente a 0.64 con TF-IDF, mostrando el potencial de las representaciones densas para mejorar el clustering en documentos.

Por otro lado, Dey **[3]** aplicó TF-IDF y Bag of Words a comentarios de Reddit para clasificarlos mediante Naïve Bayes, obteniendo un 89.75% de exactitud con TF-IDF.

Fortalezas: Radu et al. **[2]** hizo uso de representaciones vectoriales avanzadas (como Doc2Vec) que superan en rendimiento de agrupamiento a los modelos tradicionales basados en conteo de frecuencias.

Debilidades: limitación de los modelos dispersos (TF-IDF y Bag of Words) para capturar la semántica profunda y el contexto de las publicaciones en comparación con los vectores densos.

Amenazas: el ruido léxico y la informalidad en redes sociales, puede perjudicar a la vectorización de los datos.

Oportunidades: emplear representaciones densas optimizadas con algoritmos no supervisados y frameworks (**Optuna**) de búsqueda de los mejores parámetros.

D. ¿Cuál es nuestra propuesta para abordarlos?

Diversidad en la vectorización: Un módulo flexible que permita alternar entre representaciones densas y métodos dispersos.

Optimización mediante GPU: uso de hardware gráfico, si se dispone, mediante PyTorch para procesar en paralelo las representaciones densas.

Búsqueda Bayesiana: uso de Optuna para maximizar de forma automática el Silhouette Score sin intervención manual.

II. Análisis de datos

A. Descripción cualitativa y fuente de los datos raw

El conjunto de datos utilizado se encuentra en el fichero 500_Reddit_users_posts_labels.csv, en formato CSV, y contiene publicaciones de Reddit asociadas a usuarios y etiquetas de clasificación **[1]**. Cada registro incluye tres atributos: User, identificador del usuario; Post, que contiene una **lista de publicaciones** almacenadas como texto, y Label, que indica la categoría asignada al usuario. Las etiquetas presentes son Ideation, Supportive, Indicator, Behavior y Attempt, relacionadas con distintas categorías de contenido sobre conducta suicida.

B. Pre-proceso: transformación de datos de texto a un vector numérico.

El código implementa cinco métodos de representación textual: TF-IDF, Bag of Words, Sentence Transformers, Doc2Vec y características emocionales mediante pysentimiento. La unidad de análisis puede configurarse como usuario o mensaje. Por defecto, se utiliza la unidad mensaje, en la que cada publicación puede constituir una instancia independiente, heredando el usuario y su etiqueta; alternativamente, se puede utilizar la unidad usuario, uniendo sus publicaciones en un único texto.

- **TF-IDF**: se convierten los textos a minúsculas, se eliminan caracteres no contemplados por la expresión regular, se tokenizan en inglés, se eliminan palabras vacías, y se aplica stemming mediante Snowball. Después TfidfVectorizer genera una matriz dispersa con unigramas, bigramas y trigramas, con un máximo predeterminado de 5000 características, frecuencia mínima de documento de 2 y máxima de 0.9.
- **Bag of Words**: los textos se someten al mismo proceso de limpieza que TF-IDF. Después, CountVectorizer construye una matriz dispersa en la que cada característica representa un unigrama o bigrama y su valor indica la frecuencia con la que aparece en el texto. Se establece un máximo predeterminado de 10000 características.
- **Sentence Transformers**: se utiliza el modelo paraphrase-multilingual-MiniLM-L12-v2 para obtener vectores densos que representen el contenido semántico. Los textos extensos se dividen en fragmentos de 80 palabras, con un máximo predeterminado de 50 fragmentos por instancia. Sus vectores se promedian y se normalizan con la norma L_2.
- **Doc2Vec**: se entrena un modelo sobre los textos disponibles, tokenizados mediante simple_preprocess de Gensim. La configuración predeterminada genera vectores de 100 dimensiones durante 40 épocas, utilizando el método PV-DBOW (dm=0). Los vectores resultantes también se normalizan mediante la norma L_2.
- **Características emocionales**: mediante pysentimiento y su analizador de emociones en inglés, cada fragmento se representa mediante siete probabilidades: ira, asco, miedo, alegría, otras emociones, tristeza y sorpresa. Para cada instancia, se promedian las probabilidades de sus fragmentos, obteniendo un vector de siete dimensiones.

En las cinco alternativas, las etiquetas se conservan separadas del texto utilizado para generar las características, de modo que pueden emplearse posteriormente en la evaluación externa del agrupamiento. Los vectores calculados también pueden almacenarse en caché para evitar repetir la codificación de los mismos textos.

C. Descripción cuantitativa: número de instancias, instancias repetidas (con los mismos atributos), atributos, talla vocabulario, OOV, missing values, rango de los atributos, …

El conjunto de datos original contiene 500 usuarios y tres atributos: User, Post y Label. Las etiquetas se distribuyen en cinco categorías: Ideation (171 usuarios), Supportive (108), Indicator (99), Behavior (77) y Attempt (45).

Al establecer el mensaje como la unidad de análisis, se obtienen 9127 instancias. Cada una conserva el identificador del usuario y su etiqueta original. El número de mensajes por usuario presenta una media de 18.25, una mediana de 8 y un rango de 1 a 292 mensajes.

En el CSV original, no existen valores ausentes ni filas duplicadas completas. Sin embargo, tras separar los mensajes, se identifican 9107 textos distintos, con 20 apariciones adicionales debidas a repeticiones. Considerando conjuntamente los tres campos User, Label y Mensaje, **hay dos instancias duplicadas** adicionales.

La dimensionalidad y el rango de los atributos dependen de la representación textual utilizada. Observar Tabla 1.

Representación	Dimensiones	Características
Bag of Words	Hasta 10000	Frecuencia de unigramas y bigramas, valores enteros no negativos.
TF-IDF	Hasta 5000	Ponderaciones entre 0 y 1; utiliza unigramas, bigramas, y trigramas.
Sentence Transformers	384	Vectores densos normalizados mediante la norma L_2
Doc2Vec	100	Vectores densos normalizados mediante la norma L_2
Características emocionales	7	Probabilidad entre 0 y 1, correspondiente a siete categorías emocionales.

Tabla 1: Representaciones con sus respectivas dimensiones y características

III. Clustering

A. Algoritmo de clustering

```
Entrada: Matriz de datos X ∈ ℝ^(N×M), número de clústeres K,modo de inicialización, métrica de distancia d, max_iter, tol
Salida: Conjunto de clústeres S = {S₁, ..., S_K}, Centroides finales μ = {μ₁, ..., μ_K}
Inicializar centroides μ₁, μ₂, ..., μ_K ∈ ℝ^M (Aleatorio o K-Means++)
Para t = 1 hasta max_iter:
        μ^(prev) ← μ
        Para cada muestra x_i ∈ X:
                c_i ← arg min_k d(x_i, μ_k)
        Para cada clúster k ∈ {1, ..., K}:
                S_k ← {x_i ∈ X | c_i = k}
        Para cada clúster k ∈ {1, ..., K}:
                Si |S_k| > 0:
                        μ_k ← (1 / |S_k|) * Σ(x_i ∈ S_k) x_i
                Si d es Coseno (Spherical K-Means):
                        μ_k ← μ_k / ||μ_k||₂
        Si ||μ - μ^(prev)||₂ ≤ tol:
                Romper bucle
Retornar S y μ
```

Fig 1: Pseudocódigo implementación K-Means

B. Coste computacional (tiempo y memoria) mediante evidencias experimentales

El coste se documenta mediante la monitorización en tiempo real con las librerías *psutil* y *time*, cuyos resultados se exportan en diagramas de barras y líneas. El cuello de botella principal reside en el preproceso de los datos (especialmente al generar Embeddings o inferir emociones mediante modelos densos), mientras que la fase de ejecución de K-Means y la evaluación de métricas presentan un consumo marginal de segundos y un pico de memoria RAM estable. La complejidad algorítmica de nuestra implementación de K-means es $O(I \cdot K \cdot N \cdot M)$ en cada iteración. En la Figura 2 se puede apreciar un ejemplo de la monitorización de uso de memoria.

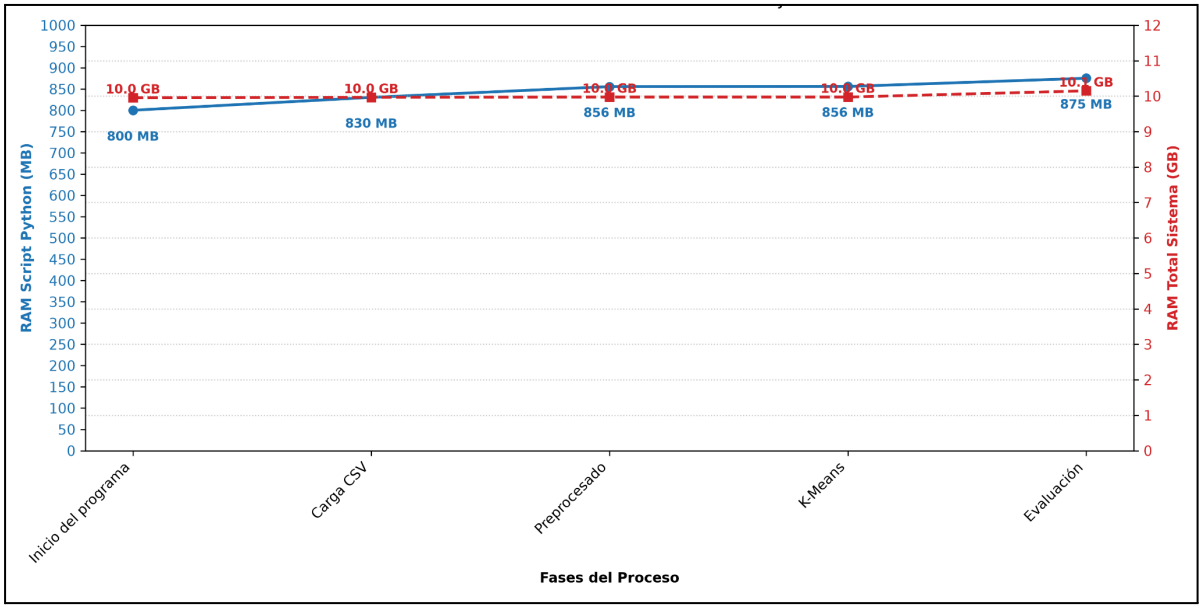


Fig 2: Evolución de uso de memoria RAM durante la fase de preprocesamiento y entrenamiento

C. ¿Qué medidas de similitud/disimilitud se han contemplado? ¿por cuál nos decantamos?

Se han implementado y evaluado tres métricas: Euclidiana, Coseno y Manhattan. Al configurar la distancia Coseno, el algoritmo opera matemáticamente como la variante Spherical K-Means. En esta modalidad, tras recalcular la posición de cada centroide, se aplica una renormalización (norma L_2) para proyectarlo sobre la superficie de una hiperesfera unitaria, lo que garantiza que el agrupamiento se haga en base a la similitud angular sin importar la magnitud vectorial.

Aunque Spherical K-Means se considera el estándar teórico para vectores de texto dispersos, la experimentación demostró que la distancia Manhattan maximiza el *Silhouette Score* utilizando el vectorizador ganador (*emotion*). Dado que este modelo genera un espacio multidimensional de probabilidades continuas, la distancia Manhattan (norma L_1) resulta superior al evaluar de forma lineal las diferencias absolutas entre cuotas emocionales, mitigando el ruido inherente a la extrema dispersión de estos datos sin las distorsiones de las métricas cuadráticas.

D. ¿Cómo elegir el número de clusters para esta tarea?

Se ha optado por un enfoque automatizado mediante Optimización Bayesiana con el algoritmo Tree-structured Parzen Estimator (TPE) **[4]**. Para implementarlo nos hemos ayudado de la librería *Optuna* **[5]**. El optimizador explora el espacio de hiperparámetros (vectorizador, métrica de distancia, inicialización y número de clusters K) utilizando el *Silhouette Score* como métrica objetivo a maximizar. Para garantizar mantener la consistencia del objetivo del modelo se estableció un límite superior de $K = 20$ evaluado a lo largo de 100 experimentos. Esta restricción evita que el algoritmo sobrefragmente los datos persiguiendo una pureza matemática artificial, forzándolo a identificar macro-patrones.

E. ¿Cómo almacenamos el modelo de clustering?

El modelo entrenado se serializa como un archivo binario .pkl mediante la librería *pickle* encapsulando un diccionario autónomo. Esta estructura de datos almacena los centroides finales, la métrica de distancia, el modo de inicialización, el identificador del preprocesamiento de texto (text_process) y el objeto vectorizador ajustado. Esto garantiza el encapsulamiento total del pipeline, independizando la fase de inferencia futura de los archivos de configuración externos

F. Proceso para asignar un cluster a una nueva instancia no vista en la fase de entrenamiento.

El diseño modular diferencia estrictamente el entrenamiento de la inferencia.

En entrenamiento: El modelo ajusta sus pesos, crea su vocabulario (en caso de usar BoW/TF-IDF/Doc2Vec) y calcula la posición de los centroides.

En inferencia: El texto nuevo se vectoriza utilizando exclusivamente el método .transform() o .infer_vector() del vectorizador congelado, mapeando la nueva instancia exactamente al mismo espacio dimensional. Posteriormente, se calcula la distancia entre este nuevo vector y los centroides fijos, asignando el clúster mediante argmin. Este mecanismo previene colisiones de dimensionalidad.

G. ¿Es versátil el software implementado? ¿Se puede extender fácilmente a otros conjuntos de datos?

El software está diseñado con una alta modularidad, haciéndolo extensible a cualquier conjunto de datos textuales. A través de parámetros por terminal y el archivo clasificador.json, el usuario puede apuntar a cualquier archivo CSV y especificar la columna objetivo sin alterar el código fuente. La arquitectura soporta de forma transparente el paso de matrices dispersas a matrices densas, permitiendo procesar nuevos dominios sin refactorización.

IV. Evaluación

A. Técnicas de evaluación

Para evaluar el modelo generado, empleamos distintas métricas de evaluación sacadas de los apuntes de la asignatura **[6]**. Estas, se dividen principalmente en dos grupos, las internas y externas.

1) Internas

Las técnicas de evaluación internas, exploran la estructura intrínseca de los datos como compromiso de dos métricas.

Cohesión: mide cuán compactos o densos son los clústeres calculando la suma de los errores cuadráticos de los puntos respecto a su centroide asignado. Ver fórmula en la Figura 3.

$$SSE(\mathcal{G}) = \sum_{i=1}^{m} SSE(\mathcal{G}_i) = \sum_{i=1}^{m} \sum_{\mathbf{x} \in \mathcal{G}_i} d^2(\mathbf{x}, \mathbf{c_i})$$

Fig 3: Formalismo teórico de la Cohesión **[6]**

Cuanto menor sea SSE, mayor será la cohesión.

Separabilidad: evalúa la distancia mínima existente entre cualquier par de centroides del modelo. Ver fórmula en la Figura 4.

$$BSS(\mathcal{G}) = \sum_{i=1}^{m} |\mathcal{G}_i| d^2(c, c_i)$$

Fig 4: Formalismo teórico de la Separabilidad **[6]**

Cuanto mayor sea BSS, mayor será la separabilidad.

Silhouette Score: calcula la media de la silueta de todas las muestras, evaluando cuán similar es un objeto a su propio clúster en comparación con otros clústeres. Ver fórmula en la Figura 5.

$$Silhouette(x_i) = \frac{b(x_i) - a(x_i)}{\max\{b(x_i), a(x_i)\}}$$

Fig 5: Formalismo teórico de la Silhouette **[6]**

Su rango va de $[-1, 1]$, donde los valores cercanos a 1 indican clústeres densos y bien separados.

2) Externas

Las métricas de evaluación externa, comparan el resultado que proporciona un algoritmo respecto de un conjunto de datos etiquetado, también se conoce como cluster vs class evaluation.

Primero, se debe calcular la matriz de contingencia. Dadas dos particiones, C (real) y G (system), contamos los pares de instancias (pair-wise comparison). Observar la matriz en la Figura 6.

- N11 = pertenecen al mismo cluster en $\mathcal{C}$ y al mismo grupo en $\mathcal{G}$
- N10 = pertenecen al mismo cluster en $\mathcal{C}$ y a distinto grupo en $\mathcal{G}$
- N01 = pertenecen a distinto grupo en $\mathcal{C}$ y al mismo grupo en $\mathcal{G}$
- N00 = pertenecen a distinto grupo en $\mathcal{C}$ y a distinto grupo en $\mathcal{G}$

		$\mathcal{C}$	
		=	≠
$\mathcal{G}$	=	N11	N01
	≠	N10	N00

Fig 6: Matriz pair-wise comparison **[6]**

Jaccard: similitud entre dos particiones basándose en los pares de elementos que coinciden en el mismo clúster en ambas particiones. Ver fórmula en la Figura 7.

$$Jaccard(\mathcal{G},\mathcal{C}) = \frac{N11}{N11 + N10 + N01}$$

Fig 7: Formalismo teórico de Jaccard **[6]**

Rand Index: porcentaje de decisiones correctas que toma el algoritmo (pares de elementos que están juntos en el mismo clúster o separados en ambos esquemas) sobre el total de pares posibles. Ver fórmula en la Figura 8.

$$RandIndex(\mathcal{G},\mathcal{C}) = \frac{N11 + N00}{N11 + N10 + N01 + N00}$$

Fig 8: Formalismo teórico de Rand Index **[6]**

Folkes-Malows: media geométrica entre la precisión y el recall basadas en los pares agrupados. Ver fórmula en la Figura 9.

$$FolkesMalows(\mathcal{G},\mathcal{C}) = \sqrt{\frac{N11}{N11 + N01} \cdot \frac{N11}{N11 + N10}}$$

Fig 9: Formalismo teórico de Folkes-Malows **[6]**

B. Baseline

Tras realizar la comparativa entre el modelo que hemos desarrollado y el de Sklearn, podemos concluir distintos puntos.

Nuestro modelo ofrece mayor versatilidad, incorporando la distancia del Coseno o el método de clasificación no supervisada K-Means tradicional (sin ++).

Asimismo, demora menos tiempo en ejecutarse y registra picos de memoria más bajos.

V. Resultados experimentales

A. Objetivo

Se evalúa un pipeline de clasificación no supervisado mediante el algoritmo K-Means o K-Means++ y diversas distancias (Manhattan, Euclídea y Coseno), así como diferentes técnicas de representación de texto (TF-IDF, BoW, Embeddings, Doc2Vec y análisis de emociones); con el fin de obtener el modelo con la combinación de parámetros más óptimo según la métrica Silhouette.

B. Anexo

1) Optimización con Optuna

Para la fase experimental se empleó Optuna **[5]**, un framework de optimización hiperparamétrica basado en la estimación de densidad por árboles. Optuna se ha convertido en un estándar de la industria y la investigación en Machine Learning gracias a su eficiencia superior frente a la búsqueda en rejilla (Grid Search) o aleatoria, podando de forma inteligente las combinaciones poco prometedoras de preprocesamiento **[7]**.

2) Rango de clusters para las pruebas

El objetivo de clasificar si una persona muestra indicios de cometer suicidio, si necesita ayuda urgente o si está ejerciendo un rol de apoyo requiere particiones claras y bien delimitadas. Mantener un k (número de clusters) bajo evita la sobrefragmentación de los datos. Partimos de que conocemos la estructura de los datos y de donde provienen **[1]**,**[8]**. Por ende, establecemos un criterio de búsqueda del k más óptimo de entre 2 y 20.

C. Conclusiones

Se han realizado 100 pruebas mediante Optuna y el modelo con mejor desempeño conforme a Silhouette ha sido utilizando el preproceso emotion (denso), K-Means++, distancia Manhattan y $k = 5$; con un resultado Silhouette de 0.7056.

Asimismo, si se quiere revisar cualquier otro método de evaluación, se puede recurrir al banco de pruebas almacenado en *"output/metricas_modelos.csv"* donde por cada modelo se almacenan distintas **métricas**.

El uso de Optuna permitió identificar de manera sistemática y eficiente que la combinación de representaciones densas (como Embeddings y emociones) junto con distancias adecuadas maximiza el *Silhouette Score*, superando la configuración manual tradicional.

VI. Conclusiones y trabajo futuro

A. Nuestra aportación respecto a los antecedentes y las soluciones que hay accesibles

Automatización y flexibilidad: un pipeline no supervisado que optimiza de forma automática la vectorización del texto y los parámetros del clúster, la evaluación y la interpretación cualitativa mediante modelos de lenguaje locales.

Algoritmo K-Means propio: se ha creado desde cero de un algoritmo K-Means más versátil.

Variedad de configuraciones: se han desarrollado distintos preprocesos, así como parámetro para el algoritmo K-Means. Todos ellos intercambiables fácilmente desde un fichero de configuración.

Optimización bayesiana: integra Optuna para explorar con alta eficiencia el espacio de configuraciones.

B. ¿Cuáles son las debilidades y las fortalezas de nuestro proyecto?

Fortalezas: Soporte para aceleración por GPU en representaciones densas, alta modularidad metodológica (intercambio de configuración rápida e intuitiva) y una evaluación rigurosa basada en métricas internas y externas frente a datos reales.

Debilidades: Vulnerabilidad ante el ruido y las expresiones informales propias de las redes sociales, además de la sensibilidad en los límites de los clústeres al trabajar con espacios vectoriales complejos.

C. Líneas de trabajo futuro

Incorporar modelos de lenguaje de gran escala (LLMs) especializados en biomedicina para refinar la extracción de embeddings textuales.

Ampliar el banco de pruebas con algoritmos de clustering alternativos para contrastar su rendimiento frente a K-Means.

VII. Bibliografía

[1] M. Gaur et al., «Dataset for: Characterization of time-variant and time-invariant assessment of suicidality on Reddit using C-SSRS», Zenodo, 2019. [En línea]. Disponible en: https://zenodo.org/records/2667859 [Accedido: 28-sep-2026].

[2] R.-G. Radu, I.-M. Radulescu, C.-O. Truica, E. S. Apostol y M. Mocanu, «Clustering Documents using the Document to Vector Model for Dimensionality Reduction», en Proceedings of the 2020 IEEE International Conference on Automation, Quality and Testing, Robotics (AQTR), 2020, pp. 1–6. [En línea]. Disponible en: https://doi.org/10.1109/AQTR49680.2020.9129967

[3] R. Dey, «Suicidal Comments Detection on Reddit Platform», Medium, 11 de julio de 2021. [En línea]. Disponible en: https://medium.com/@rouhinadey98/suicidal-comments-detection-on-reddit-platform-50fb4b2d080f

[4] J. Bergstra, R. Bardenet, Y. Bengio, and B. Kégl, "Algorithms for Hyper-Parameter Optimization." NIPS 2011 [En línea]. Disponible en: https://papers.nips.cc/paper_files/paper/2011/file/86e8f7ab32cfd12577bc2619bc635690-Paper.pdf [Accedido: 19-oct-2026]

[5] Optuna, "Optuna - a Hyperparameter Optimization Framework," *Optuna*. https://optuna.org/ [Accedido: 19-Oct-2026].

[6] A. Perez, «Clustering», *Minería de Datos del grado en Ingeniería Informática de Gestión y Sistemas eGela EHU*, 2026. [En línea]. Disponible en: https://egela.ehu.eus/pluginfile.php/12387763/mod_resource/content/4/DM_slides_2_Clustering_Handout.pdf. [Accedido: 12-oct-2026].

[7] T. Akiba, S. Sano, T. Yanase, T. Ohta y M. Koyama, «Optuna: A Next-generation Hyperparameter Optimization Framework», 2019. [En línea]. Disponible en: https://arxiv.org/abs/1907.10902 [Accedido: 20-oct-2026].

[8] M. Gaur, A. Alambo, J. P. Sain, K. Thirunarayan, A. S. Gundlapalli y A. P. Sheth, «Knowledge-aware Assessment of Severity of Suicide Risk for Early Intervention», en *Companion Proceedings of The 2019 World Wide Web Conference (WWW '19)*, 2019. [En línea]. Disponible en: https://dl.acm.org/doi/pdf/10.1145/3308558.3313698 [Accedido: 19-oct-2026].